

Why Attention Needs Position

Proof, with real GPT-2 weights, that attention is blind to order · video 2:45

This lesson answers one question: why do transformers need position information at all? **Attention** is a weighted average whose weights come from dot products, so shuffling the input just shuffles the output (**permutation equivariance**). With GPT-2's real weights and its position embeddings switched off, “dog bites man” and “man bites dog” give identical vectors. Position embeddings, and to a lesser degree the causal mask, bring order back; GPT-2's learned table, however, stops at 1,024 positions.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 6).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. Attention is blind to order	▶ 0:08-0:42	2	☐
2. The experiment	▶ 0:42-1:22	2	☐
3. The causal mask leaks order	▶ 1:22-1:46	2	☐
4. What GPT-2 learned, and its limit	▶ 1:46-2:43	2	☐

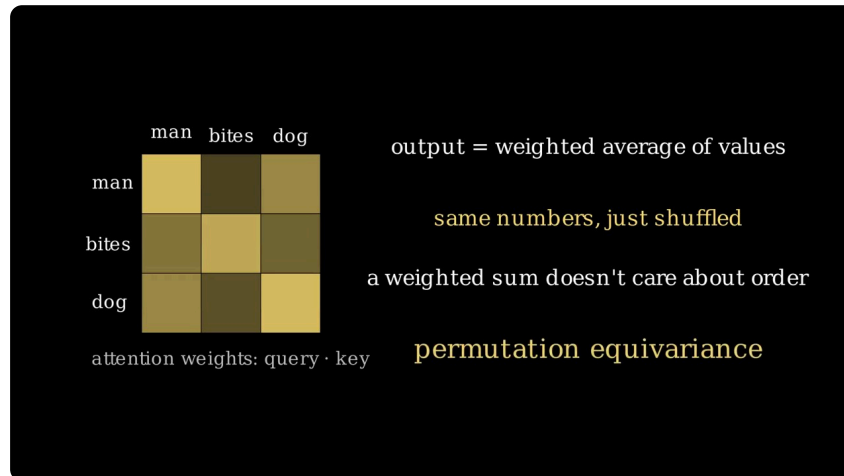
Before you start: How LLMs Work episodes 4–6 (position, attention) and Foundations F02 (dot product, cosine similarity). Code: `code/d04_position` (GPT-2 small, about 500 MB).

CONCEPT 1 OF 4

Attention is blind to order

▶ Video 0:08–0:42

Dog bites man and *man bites dog*: same three words, opposite news. But attention has no idea of order. Each token's output is a **weighted average of value vectors**, and the weights come from **dot products** between queries and keys. Shuffle the input tokens, and you get the same dot products, just shuffled; a weighted sum does not care about order. This property is called **permutation equivariance**.



▶ 0:41 Shuffle the tokens and the attention weights are the same numbers, just shuffled.

KEY IDEA

Without extra information, attention treats a sentence as a bag of tokens: reorder the input, and the outputs are simply reordered.

Check 1: Attention is blind to order

Answer all 2 questions before looking at the answers. Pass mark: all correct.

1.1 SHORT ANSWER

In your own words: why does reordering the inputs only reorder the outputs of attention?

1.2 TRUE OR FALSE

“Permutation equivariance means the outputs are identical for every order.”

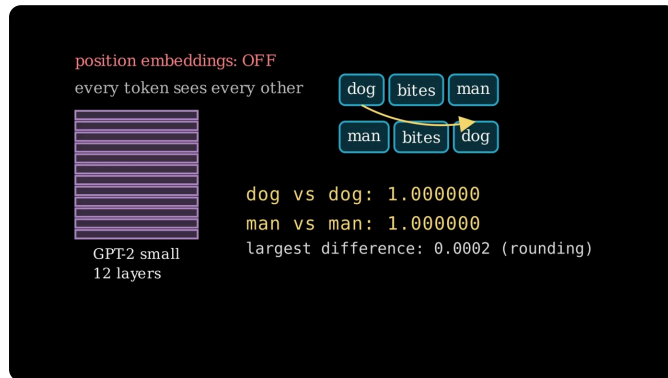
True ☐ False ☐

CONCEPT 2 OF 4

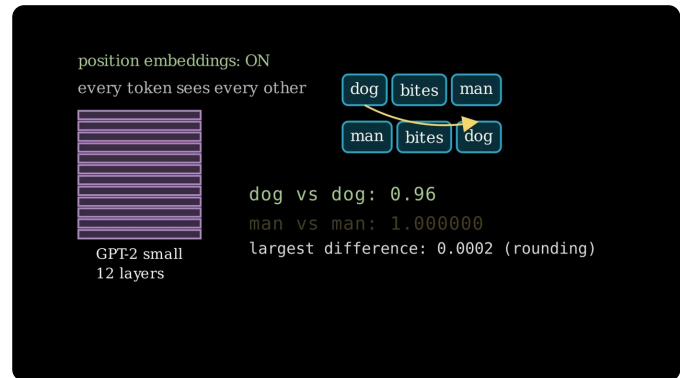
The experiment

▶ Video 0:42-1:22

Take GPT-2 small, all 12 layers. Switch off its position embeddings, let every token see every other, and feed in both sentences. The final vector for *dog* in the first sentence and in the second: cosine similarity **1.000000**. The same for *man*. The biggest difference anywhere is 0.0002, just rounding. The model cannot tell who bit whom.



▶ 1:08 GPT-2, position embeddings off, every token seeing every other: dog vs dog 1.000000.



▶ 1:20 Position embeddings back on: dog vs dog drops to 0.96.

Switch the position embeddings back on, and dog in position 1 is no longer the same as dog in position 3: the similarity drops to **0.96**. Now order can matter.

KEY IDEA

Position information is the only thing that lets the model tell “dog bites man” from “man bites dog”.

Check 2: The experiment

Answer all 2 questions before looking at the answers. Pass mark: all correct.

2.1 MULTIPLE CHOICE

With position embeddings off and no mask, what would GPT-2 do with “the cat chased the mouse” and “the mouse chased the cat”?

- A. Clearly different vectors for “cat”
- B. Identical vectors for “cat”, just reordered
- C. An error
- D. Random vectors

2.2 HANDS-ON CODE

Try it yourself. Using `last_layer` from `position.py`, compare “cat” (token 1 in the first sentence, token 4 in the second) with positions off and on. Why is the “on” number so close to 1, unlike dog's 0.96?

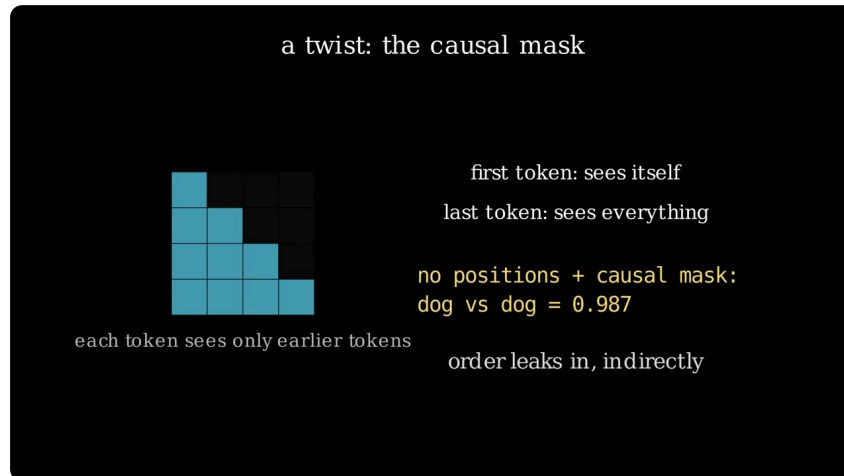
```
a = tok(" the cat chased the mouse", return_tensors="pt").input_ids
b = tok(" the mouse chased the cat", return_tensors="pt").input_ids
for pos in (False, True):
    oa, ob = last_layer(a, pos), last_layer(b, pos)
    print(pos, torch.nn.functional.cosine_similarity(oa[1], ob[4], dim=0).item())
```

CONCEPT 3 OF 4

The causal mask leaks order

▶ Video 1:22-1:46

Models like GPT use a **causal mask**: each token sees only the tokens before it. That alone leaks some order: the first token sees only itself, the last sees everything. With the mask on and no position embeddings, the two dogs are no longer identical: **0.987**. Some models are trained this way, without explicit positions, but it is an indirect signal.



▶ 1:45 With a causal mask and no position embeddings, the two dogs differ: 0.987.

KEY IDEA

A causal mask breaks the symmetry a little; explicit position information makes order a first-class input.

Check 3: The causal mask leaks order

Answer all 2 questions before looking at the answers. Pass mark: all correct.

3.1 CALCULATION

In a 4-token causal mask (as on screen), how many of the 16 query-key pairs are allowed?

.....

3.2 SHORT ANSWER

With a causal mask, why does the first token's output depend on its position?

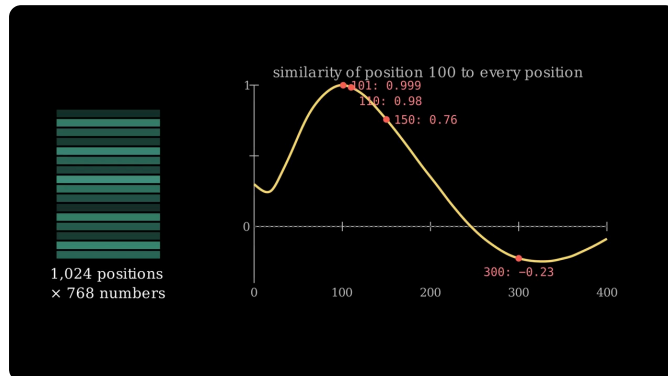
.....

CONCEPT 4 OF 4

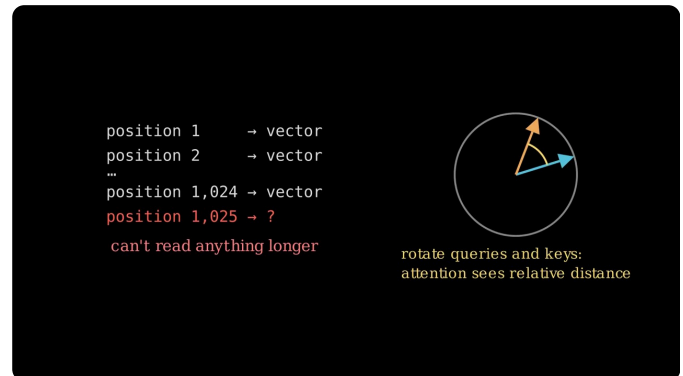
What GPT-2 learned, and its limit

▶ Video 1:46-2:43

GPT-2 learned one vector per position: **1,024** of them, each with 768 numbers, from scratch. A smooth pattern emerges: position 100 is almost identical to 101 (0.999), still very close to 110 (0.98), less so to 150 (0.76), and at 300 the similarity is negative (−0.23).



▶ 2:09 Similarity of GPT-2's position vector 100 to every position 0-400: smooth, and negative far away.



▶ 2:28 A learned table stops at 1,024; rotating queries and keys encodes relative distance.

But a learned table has a hard limit: GPT-2 has no vector for position 1,025, so it cannot read anything longer. Modern models encode position by **rotating** the queries and keys, so attention sees **relative distance** and the range can be stretched (next episode).

KEY IDEA

Learned absolute positions work, but are capped by the table size and say nothing directly about distance between tokens.

Check 4: What GPT-2 learned, and its limit

Answer all 2 questions before looking at the answers. Pass mark: all correct.

4.1 CALCULATION

How many numbers are in GPT-2's position-embedding table?

4.2 TRUE OR FALSE

"GPT-2 can process a 2,000-token document in one pass."

True ☐ False ☐

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. Attention is blind to order · rewatch ▶ 0:08-0:42

1.1 Each output is computed from dot products with all tokens and a weighted sum over them; neither operation depends on where a token sits in the list. Sums and dot products treat their inputs as a set.

1.2 False. The outputs are the same vectors in a different order: each token's own output does not change, only its position in the list.

2. The experiment · rewatch ▶ 0:42-1:22

2.1 B. Checked by running it: cosine similarity 1.000000.

2.2 Off: 1.000000. On: 0.999767, different, but barely by this measure. GPT-2's final vectors share a few very large dimensions, so cosine similarity between them is high overall; it is a blunt measure here. The "largest difference" printed by position.py shows the change more clearly.

3. The causal mask leaks order · rewatch ▶ 1:22-1:46

3.1 10. $1 + 2 + 3 + 4 = 10$: each token sees itself and all earlier tokens.

3.2 It can only attend to itself, while a later token can attend to several; so the set of tokens it mixes depends on where it sits. Position changes what each token can see, even without position embeddings.

4. What GPT-2 learned, and its limit · rewatch ▶ 1:46-2:43

4.1 786,432. 1,024 positions $\times$ 768 numbers.

4.2 False. Its learned position table has 1,024 entries; there is no vector for position 1,025 or beyond.